CS Interview Notes Top 100

Questions, answers, and common pitfalls

A curated set of 100 foundational computer science interview questions. These are original study notes, not an official ranking or a guarantee of interview coverage. Language and database behavior can depend on implementation and configuration.

How to study

Read a topic, answer from memory, explain an example, and revisit the questions you missed. Aim for a clear 30-60 second explanation before exploring a follow-up.

Ten topics

01 Data Structures and Complexity (Questions 1-10)

02 Algorithms and Problem Solving (Questions 11-20)

03 Operating Systems and Concurrency (Questions 21-30)

04 Databases and Structured Query Language (Questions 31-40)

05 Object-Oriented Design (Questions 41-50)

06 Computer Networks and Web Fundamentals (Questions 51-60)

07 Distributed Systems and System Design (Questions 61-70)

08 Python and Java Fundamentals (Questions 71-80)

09 Security Fundamentals (Questions 81-90)

10 Testing, Git, and Engineering Practice (Questions 91-100)

Prepared for Manasa Rajagopal Madabushi
Study edition - 7 November 2026

01 Data Structures and Complexity

Questions 1-10

1. What do Big O, Big Omega, and Big Theta mean?

Big O is an asymptotic upper bound, Big Omega a lower bound, and Big Theta a tight bound on growth. State whether you are analyzing worst-case, average-case, or another case.

Remember: O(n) is not automatically a worst-case statement; the analyzed case must be specified.

2. How do time complexity and space complexity differ?

Time complexity describes how work grows with input size. Space complexity describes memory growth. Distinguish auxiliary memory from total memory, and count recursive call frames when estimating space.

Remember: A recursive traversal may use O(h) stack space even when it allocates no explicit collection.

3. What is amortized analysis?

Amortized analysis bounds the average cost per operation across a sequence, without assuming a probability distribution. A dynamic array occasionally performs an expensive resize, but repeated appends can still take amortized constant time.

Remember: One append can cost O(n); a sequence of n appends typically costs O(n) overall.

4. When should you choose an array over a linked list?

Arrays provide constant-time indexed access and good locality. Linked lists support constant-time insertion or deletion when the relevant node and links are already known. Finding a position in a linked list still takes linear time.

Remember: Do not claim all linked-list insertions are O(1); locating the insertion point matters.

5. How do stacks and queues differ?

A stack removes the most recently added element first. A queue removes the earliest added element first. Stacks support parsing and depth-first traversal; queues support scheduling and breadth-first traversal.

Remember: Stack: last in, first out. Queue: first in, first out.

6. How does a hash table work?

A hash function maps a key to a bucket or starting position. Collisions are handled through techniques such as chaining or open addressing. Expected constant-time operations depend on hashing assumptions and controlled occupancy.

Remember: Worst-case lookup can be O(n); resizing and load factor affect practical performance.

7. What makes a binary search tree efficient?

Each node separates smaller and larger keys according to a consistent ordering policy. Search takes O(h), where h is tree height. Balanced trees keep height logarithmic; an unbalanced tree can become a chain.

Remember: Balanced search: O(log n). Degenerate search: O(n).

8. What is a heap, and when is it useful?

A binary heap maintains a partial order between parents and children. It supports efficient access to the minimum or maximum element and is commonly used for priority queues, scheduling, and top-k problems.

Remember: Peek: O(1). Insert and remove root: O(log n). Bottom-up heap construction: O(n).

9. What is a trie?

A trie stores keys along paths labeled by characters or other symbols. It is useful for prefix lookup and autocomplete. Lookup depends on key length rather than the number of stored keys, but memory overhead can be substantial.

Remember: Search is O(L) with constant-time child access, where L is key length.

10. How are graphs represented?

An adjacency list stores neighbors for each vertex and is efficient for sparse graphs. An adjacency matrix stores all possible vertex pairs and provides direct edge lookup, but uses quadratic space.

Remember: Adjacency list: O(V + E) storage. Matrix: O(V squared) storage.

02 Algorithms and Problem Solving

Questions 11-20

11. How does binary search work?

Binary search repeatedly discards half of an ordered search interval. It also applies to monotonic predicates, such as finding the first feasible capacity. Define interval boundaries and prove that each iteration shrinks the interval.

Remember: O(log n) comparisons; test empty inputs, duplicates, and boundary positions.

12. When are two pointers useful?

Two pointers track positions that move according to a useful invariant. Examples include finding pairs in a sorted array, partitioning values, and comparing characters from opposite ends. Sorting first may add O(n log n) work.

Remember: Explain why moving a pointer cannot discard a valid answer.

13. What is the sliding-window technique?

A sliding window maintains information about a contiguous range while adjusting its boundaries. It works when the maintained condition supports efficient updates and valid pointer movement. A fixed-size maximum-sum window is a common example.

Remember: Some variable-size sum problems break the usual shrinking rule when negative values are present.

14. When should you use breadth-first search or depth-first search?

Breadth-first search explores vertices by distance from the start and finds shortest paths in unweighted graphs. Depth-first search explores branches deeply and supports cycle detection, component discovery, and dependency analysis.

Remember: With adjacency lists, both run in O(V + E). Mark visited vertices appropriately.

15. What is topological sorting?

A topological ordering places every directed edge's source before its destination. It exists only for directed acyclic graphs. Kahn's algorithm repeatedly removes zero-indegree vertices; a depth-first approach uses finishing order.

Remember: If Kahn's algorithm processes fewer than V vertices, the graph contains a directed cycle.

16. How do common shortest-path algorithms differ?

Breadth-first search handles unit-weight edges. Dijkstra's algorithm requires nonnegative edge weights. Bellman-Ford supports negative edges and can detect a reachable negative-weight cycle. Choose based on graph constraints rather than familiarity.

Remember: Dijkstra with an adjacency list and binary heap is commonly O((V + E) log V).

17. How do merge sort and quicksort compare?

Merge sort guarantees O(n log n) time and is stable when ties are merged consistently. Quicksort partitions around pivots, usually performs well, and has O(n squared) worst-case time. Memory usage depends on the implementation.

Remember: Array merge sort commonly needs O(n) auxiliary storage; quicksort also uses recursion space.

18. What is dynamic programming?

Dynamic programming solves related subproblems and reuses their results. Define the state, transitions, base cases, and evaluation order. Memoization computes needed states on demand; tabulation fills states in an explicit order.

Remember: Complexity is roughly the number of states multiplied by the work per transition set.

19. When is a greedy algorithm correct?

A greedy algorithm chooses a locally attractive option at each step. Correctness needs a proof, often using an exchange argument or a structural property. A choice that works for one input family may fail for another.

Remember: Coin-change greedy works for some coin systems, but not for arbitrary denominations.

20. How does backtracking differ from dynamic programming?

Backtracking explores candidate choices and undoes them when a branch fails. Dynamic programming reuses equivalent subproblem results. Backtracking is useful for combinations and constraint problems; memoization can sometimes reduce repeated search.

Remember: Pruning improves search but does not automatically make an exponential problem polynomial.

03 Operating Systems and Concurrency

Questions 21-30

21. What is the difference between a process and a thread?

A process usually has its own virtual address space and resources. Threads within a process share much of that state but have separate execution stacks and registers. Shared memory makes communication easier and synchronization necessary.

Remember: A process can contain many threads; a thread is not an independent address space.

22. What happens during a context switch?

The operating system saves execution state for one task and restores another task's state. Costs include scheduling work and possible disruption to caches or address-translation state. Exact overhead depends on the hardware and operating system.

Remember: A mode switch into the kernel is not necessarily a task context switch.

23. How do concurrency and parallelism differ?

Concurrency means multiple tasks can make progress during overlapping periods. Parallelism means tasks execute simultaneously. A single core can interleave concurrent tasks; multiple execution units can run tasks in parallel.

Remember: Asynchronous input/output can improve concurrency without creating CPU parallelism.

24. What are a race condition and a data race?

A race condition occurs when correctness depends on an uncontrolled ordering of events. A data race involves conflicting memory accesses without the required synchronization under the language's memory model. Not every race condition is a data race.

Remember: Fix the invariant and synchronization boundaries, not only the symptom observed in one run.

25. How do a mutex and a semaphore differ?

A mutex protects exclusive access to a resource and usually has ownership rules. A semaphore maintains a count of available permits and can coordinate access to a limited resource pool or signal events.

Remember: A binary semaphore and a mutex can have different ownership and priority behavior.

26. What are the necessary conditions for deadlock?

The classic conditions are mutual exclusion, hold and wait, no preemption, and circular wait. Breaking one condition can prevent deadlock in the relevant model. Consistent lock ordering is a common practical prevention technique.

Remember: A timeout helps recovery but does not prove that deadlock cannot occur.

27. How do deadlock, livelock, and starvation differ?

In deadlock, tasks wait indefinitely for one another. In livelock, tasks keep reacting without useful progress. In starvation, a task is repeatedly denied the resources or scheduling time it needs.

Remember: Fair scheduling and bounded retries can help, but the right remedy depends on the cause.

28. What are virtual memory and paging?

Virtual memory gives a process an address-space abstraction. Paging translates fixed-size virtual pages to physical frames. A page fault occurs when an access needs operating-system intervention, such as loading a page or rejecting an invalid access.

Remember: A page fault does not always require disk input/output; some are resolved in memory.

29. How do stack memory and heap memory differ?

A thread's stack usually holds call frames and automatically scoped execution data. A heap supports allocations with lifetimes not tied directly to one call. Exact object placement depends on language semantics, compiler optimizations, and runtime design.

Remember: Do not assume every local variable lives on the stack or every object is heap allocated.

30. Why must condition-variable waits check a predicate in a loop?

Waiting releases an associated mutex and later reacquires it. After waking, the condition may no longer hold, and some systems allow spurious wakeups. Recheck the shared-state predicate while holding the mutex before continuing.

Remember: The condition variable signals possible state changes; the predicate represents the actual condition.

04 Databases and Structured Query Language

Questions 31-40

31. How do relational and nonrelational databases differ?

Relational databases organize data into relations and support constraints and relational operations. Nonrelational systems use models such as documents, key-value pairs, or graphs. Choose based on access patterns, consistency needs, and operational requirements.

Remember: Nonrelational does not mean schema-free, and relational does not mean impossible to scale horizontally.

32. What are primary, foreign, and candidate keys?

A candidate key is a minimal set of attributes that uniquely identifies a row. One candidate key is chosen as the primary key. A foreign key enforces a relationship with a referenced unique or primary key, subject to database rules.

Remember: A composite key uses multiple attributes; a surrogate key is an artificial identifier.

33. What do normalization and denormalization achieve?

Normalization reduces redundancy and update anomalies by organizing relations around dependencies. Denormalization intentionally duplicates or precomputes data to improve specific reads. It adds storage and synchronization costs and should follow measured access requirements.

Remember: Third normal form and Boyce-Codd normal form address functional dependencies with different strictness.

34. What do the ACID properties mean?

Atomicity makes a transaction all-or-nothing. Consistency concerns preserving defined invariants. Isolation controls interactions between concurrent transactions. Durability means committed changes survive failures within the database's documented guarantees and configuration.

Remember: Database consistency in ACID is different from replica consistency in distributed systems.

35. What do isolation levels prevent?

Isolation levels define which concurrent anomalies are allowed. Read committed prevents dirty reads; stronger levels offer stronger guarantees. Serializable aims for results equivalent to a serial execution, but transactions may need to retry after conflicts.

Remember: Exact behavior differs by database; PostgreSQL repeatable read uses a transaction snapshot.

36. How does a database index improve performance?

An index stores a structure that helps locate rows without scanning all data. B-tree indexes support ordered access; other indexes suit different operators. Indexes add write overhead and consume space, so usefulness depends on query patterns.

Remember: An index is not guaranteed to be chosen; selectivity, ordering, and cost estimates matter.

37. How do inner and outer joins differ?

An inner join returns matching row combinations. A left outer join also preserves unmatched rows from the left side, filling right-side fields with nulls. A full outer join preserves unmatched rows from both sides.

Remember: Filtering right-side columns in WHERE can remove the null-extended rows of a left join.

38. How do WHERE, GROUP BY, HAVING, and window functions differ?

WHERE filters input rows. GROUP BY forms groups for aggregation, and HAVING filters those groups. Window functions calculate across related rows without collapsing them into one output row per group.

Remember: Use a subquery or another query layer when filtering a window-function result requires it.

39. How should null values be handled in SQL?

Null represents an unknown or absent value, and comparisons can produce unknown rather than true or false. Use IS NULL or IS NOT NULL for null tests. Many aggregates ignore null inputs; COUNT(*) counts rows.

Remember: NULL = NULL is not true in ordinary SQL equality; use database-specific null-safe comparison when needed.

40. How would you investigate a slow database query?

Inspect the execution plan, actual row counts, filters, joins, sorting, and index use. Check statistics, lock waits, and unnecessary data transfer. Measure changes on representative data before adding indexes or rewriting the query.

Remember: EXPLAIN ANALYZE executes the statement; be careful with statements that modify data.

05 Object-Oriented Design

Questions 41-50

41. What are the main ideas of object-oriented programming?

Encapsulation groups state and behavior behind a boundary. Abstraction exposes relevant capabilities while hiding details. Inheritance derives behavior or types from others. Polymorphism lets different implementations be used through a shared interface.

Remember: Explain the ideas with one concrete design rather than only listing the four terms.

42. How do abstraction and encapsulation differ?

Abstraction decides what a client needs to know. Encapsulation controls how implementation details and state are accessed. An interface can express an abstraction, while a class boundary can protect the state that implements it.

Remember: Private fields alone do not guarantee a well-designed abstraction or a valid object invariant.

43. When should composition be preferred over inheritance?

Composition builds behavior by delegating to other objects. It is often more flexible when capabilities vary independently. Inheritance suits a genuine substitutable relationship, but can couple subclasses to superclass implementation details.

Remember: Ask whether a subclass can honor every contract expected of the base type.

44. What is the difference between an interface and an abstract class?

An interface defines a type-level contract. An abstract class can provide shared implementation and instance state. In Java, a class can implement multiple interfaces but directly extend only one class; interfaces can also provide certain method implementations.

Remember: Interfaces are not limited to abstract methods; avoid outdated blanket claims.

45. How do overloading and overriding differ?

Overloading defines operations with the same name but different parameter signatures under a language's rules. Overriding replaces inherited instance behavior in a subtype. Overload resolution and runtime dispatch happen at different stages and vary by language.

Remember: Return type alone is not enough to overload a Java method.

46. What does the Liskov substitution principle require?

A subtype should be usable wherever its base type is expected without breaking the client's correctness assumptions. It should preserve the promised behavior, including relevant invariants and constraints on inputs and outputs.

Remember: A subclass that rejects inputs accepted by the base class may violate substitutability.

47. What is the dependency inversion principle?

High-level policy should depend on abstractions rather than concrete low-level details. Details implement those abstractions. This reduces coupling and makes substitutions easier, especially when dependencies are supplied rather than constructed inside the policy.

Remember: Dependency injection is one technique for wiring dependencies, not the principle itself.

48. When are factory and strategy patterns useful?

A factory centralizes or abstracts object creation. A strategy encapsulates an interchangeable algorithm or policy. For example, a factory can construct payment handlers, while a strategy selects how a fee is calculated.

Remember: Introduce a pattern to manage real variation, not merely to add classes.

49. What are the benefits and limits of immutability?

Immutable values cannot change after construction, which simplifies reasoning, sharing, and safe reuse. Updates create new values or structures. Immutability can increase allocation costs, and a read-only reference does not guarantee that the referenced object is immutable.

Remember: In Java, final prevents reassignment of a reference; it does not freeze the object.

50. How do you design a useful exception boundary?

Catch errors where you can recover, add context, translate them into a meaningful contract, or report them once. Release resources reliably and preserve the underlying cause when wrapping. Avoid silently swallowing unexpected errors.

Remember: Distinguish anticipated validation failures from infrastructure failures and programming defects.

06 Computer Networks and Web Fundamentals

Questions 51-60

51. How do TCP and UDP differ?

Transmission Control Protocol provides a reliable, ordered byte stream with flow and congestion control. User Datagram Protocol sends datagrams without those delivery guarantees. Applications can build reliability over UDP when their use case requires it.

Remember: TCP preserves byte order, not application message boundaries; UDP is not automatically faster in every workload.

52. What do network layers accomplish?

Layers separate responsibilities such as link delivery, routing, transport, and application semantics. The seven-layer Open Systems Interconnection model is a conceptual reference; the Internet protocol suite groups responsibilities differently.

Remember: Be able to place Ethernet, Internet Protocol, TCP, and HTTP in their respective roles.

53. What happens when you open an HTTPS website?

The client resolves the hostname, establishes the needed transport and security connection, sends an HTTP request, receives a response, and renders resources. Cached data and reused connections can skip steps; HTTP/3 uses QUIC over UDP.

Remember: Not every request performs a fresh DNS lookup, TCP handshake, or TLS handshake.

54. How does DNS resolution work?

The Domain Name System maps names to records. A client often asks a recursive resolver, which uses cached results or follows delegations to authoritative servers. Different record types represent addresses, aliases, mail routing, and other information.

Remember: Time-to-live values guide caching; DNS does not simply translate every name into one fixed address.

55. What does a TCP three-way handshake establish?

The endpoints exchange SYN, SYN-ACK, and ACK segments to establish connection state and synchronize initial sequence numbers. This allows subsequent data to be tracked and acknowledged as part of the byte stream.

Remember: The TCP handshake alone does not provide encryption or authenticate the application server.

56. What does TLS protect?

Transport Layer Security can provide confidentiality, integrity, and peer authentication for data in transit. Certificate validation checks the peer's identity against trusted authorities and the intended hostname, subject to the protocol and trust configuration.

Remember: HTTPS does not make application code secure or protect data after an endpoint decrypts it.

57. Which HTTP methods are safe or idempotent?

A safe method requests essentially read-only semantics. An idempotent method has the same intended server effect when repeated as when executed once. GET is safe and idempotent; PUT and DELETE are idempotent but not safe.

Remember: Repeated DELETE requests can return different status codes and still be idempotent.

58. How do common HTTP status codes differ?

Successful responses are generally 2xx, redirects 3xx, client errors 4xx, and server errors 5xx. Typical examples include 201 for creation, 400 for a bad request, 401 for missing or invalid authentication, and 403 for refusal.

Remember: A 404 can hide a resource's existence; status codes must be interpreted with the API contract.

59. How do cookies, sessions, and tokens relate?

A cookie is a browser storage and transport mechanism. A session associates requests with server-side state, often using an identifier in a cookie. A token carries or references credentials or claims and may also be stored in a cookie.

Remember: Cookies and tokens are not mutually exclusive alternatives; storage choice affects security risks.

60. What do a reverse proxy and a load balancer do?

A reverse proxy receives requests on behalf of backend servers. A load balancer distributes traffic across targets; one component may do both. They can also handle routing, health checks, connection reuse, or TLS termination.

Remember: Layer-four balancing and layer-seven routing make decisions using different protocol information.

07 Distributed Systems and System Design

Questions 61-70

61. What is the difference between vertical and horizontal scaling?

Vertical scaling increases a machine's resources. Horizontal scaling adds machines or instances. Horizontal scaling can improve capacity and resilience but introduces coordination, partitioning, load balancing, and consistency concerns.

Remember: More instances do not fix a shared bottleneck such as one overloaded database.

62. What is the CAP theorem actually about?

In a distributed shared-data system experiencing a network partition, it is impossible to guarantee both linearizable consistency and the theorem's availability condition for every request. This is not a universal instruction to choose any two properties permanently.

Remember: CAP availability is a formal response guarantee, not an uptime percentage or a latency target.

63. How do replication and sharding differ?

Replication keeps copies of data for resilience or read capacity. Sharding partitions data across nodes to distribute storage and work. Systems often combine them, with multiple replicas for each shard.

Remember: Replication does not eliminate coordination costs, and sharding makes cross-shard operations harder.

64. When does caching help, and what can go wrong?

Caching reuses previously computed or fetched results. It can reduce latency and backend load but introduces staleness, invalidation, eviction, and stampede risks. Define ownership, expiry, and failure behavior for each cached value.

Remember: A high hit rate is not enough if misses or stale values violate the product's requirements.

65. How do synchronous and asynchronous communication differ?

Synchronous calls require the caller to wait for a response. Asynchronous communication lets work continue or complete later through messages, events, or callbacks. It improves decoupling but adds retry, ordering, visibility, and failure-handling complexity.

Remember: Asynchronous processing does not automatically guarantee eventual success or exactly-once effects.

66. How should a service handle retries safely?

Set deadlines, retry only suitable failures, use bounded backoff with jitter, and prevent duplicate side effects. An idempotency key can associate repeated requests with one logical operation and its stored result.

Remember: A timeout does not prove the server did nothing; it may have completed the operation.

67. What does eventual consistency mean?

When updates stop and communication and reconciliation succeed, replicas eventually converge according to the system's rules. The model alone does not specify a convergence deadline or guarantee that every read sees the newest write.

Remember: Ask about read-your-writes, conflict resolution, and freshness requirements rather than using the label alone.

68. Why are rate limiting and backpressure different?

Rate limiting caps accepted activity over time or by quota. Backpressure signals that downstream capacity is constrained so producers slow or stop. Both protect a system, but they operate on different control signals.

Remember: A token bucket permits bounded bursts; a fixed window can allow bursts around window boundaries.

69. How do logs, metrics, and traces complement one another?

Logs record events with context. Metrics summarize quantities over time. Traces connect work across requests and services. Correlation identifiers and carefully chosen labels make these signals useful for diagnosing latency and failures.

Remember: High-cardinality metric labels can create serious storage and operational costs.

70. How would you begin designing a URL shortener?

Clarify traffic, retention, redirect latency, and abuse requirements. Define identifier generation, a mapping store, redirect behavior, caching, and analytics. Explain collisions, expiration, availability, and how the design grows under load.

Remember: Start with requirements and a simple workable design before adding distributed components.

08 Python and Java Fundamentals

Questions 71-80

71. What is the difference between equality and identity in Python?

Equality compares values according to the type's comparison behavior. Identity asks whether references point to the same object. Use == for value comparisons and is for identity checks such as testing against None.

Remember: Do not use is to compare strings or numbers merely because one example happens to work.

72. How do Python lists, tuples, sets, and dictionaries differ?

Lists are mutable sequences; tuples are immutable sequences. Sets store distinct hashable elements. Dictionaries map hashable keys to values and preserve insertion order in modern Python. Choose the structure that expresses the needed operations.

Remember: An immutable tuple can still refer to a mutable object; tuples are not always hashable.

73. What is the difference between a shallow copy and a deep copy?

A shallow copy creates a new outer container while preserving references to nested objects. A deep copy recursively copies supported contents and tracks already copied objects to handle sharing and cycles.

Remember: Mutating a nested list can affect both shallow copies; rebinding an outer element is different.

74. What are Python iterators and generators?

An iterator supplies successive values through the iteration protocol. A generator is a convenient way to implement that protocol using suspended execution and yield. Values are produced on demand instead of necessarily storing the entire result.

Remember: Generators can be exhausted and may retain resources or references until closed or collected.

75. Why are mutable default arguments risky in Python?

Default argument expressions are evaluated when a function is defined, not on each call. A default list or dictionary can therefore be shared across calls. Use a sentinel such as None and create a fresh object inside when needed.

Remember: Shared defaults are sometimes intentional, but should not happen accidentally.

76. What does Python's global interpreter lock imply?

In traditional global-interpreter-lock-enabled CPython builds, only one thread executes Python bytecode at a time within an interpreter. Threads can still overlap blocking input/output, and native extensions may release the lock. Free-threaded builds require separate consideration.

Remember: The lock is implementation- and configuration-dependent; it is not a universal Python language guarantee.

77. What are the JVM, JDK, and bytecode?

The Java Virtual Machine executes Java bytecode and provides runtime services. The Java Development Kit supplies development tools such as the compiler along with a runtime. Java source is commonly compiled into class files containing bytecode.

Remember: A runtime can interpret and just-in-time compile code; Java execution is not only interpretation.

78. Is Java pass-by-value or pass-by-reference?

Java passes argument values. For an object argument, the copied value is a reference to the object. A method can mutate that object through the reference, but reassigning its parameter does not reassign the caller's variable.

Remember: Swapping two parameter references inside a method does not swap the caller's variables.

79. What is the Java equals and hashCode contract?

Equality must obey its documented equivalence rules, and equal objects must have equal hash codes. Unequal objects can share a hash code. Hash-based collections depend on these contracts and on keys remaining stable while stored.

Remember: Override hashCode consistently when overriding equals; avoid mutable state in hash keys.

80. How do garbage collection and resource management differ?

Garbage collection reclaims memory for unreachable objects according to runtime rules. It does not guarantee timely release of external resources such as files, locks, or sockets. Use structured cleanup, such as Java try-with-resources or Python context managers.

Remember: An object can be reachable but useless, causing a logical memory leak.

09 Security Fundamentals

Questions 81-90

81. How do authentication and authorization differ?

Authentication establishes who or what a requester is. Authorization decides which operations that identity may perform on which resources. Both checks need to happen at appropriate trust boundaries, including server-side object access checks.

Remember: A valid login does not grant access to another user's records.

82. How do hashing, encryption, and encoding differ?

Hashing computes a digest without a decryption operation. Encryption protects confidentiality using keys and supports decryption. Encoding changes representation for transport or storage. Base64 is encoding, not encryption.

Remember: Cryptographic hashes are not suitable password storage by themselves; use a password-hashing scheme.

83. How should passwords be stored?

Use a dedicated password-hashing function with an appropriate work factor and a unique salt per password. Restrict access to stored hashes and protect recovery flows. Parameters should follow current guidance and the deployment's resource constraints.

Remember: Never store plaintext passwords or use fast general-purpose hashing alone.

84. How do parameterized queries prevent SQL injection?

Parameterized queries separate executable SQL structure from data values, so supplied values are not interpreted as SQL syntax. Use allowlists for identifiers or structural options that cannot be bound as values, and grant minimal database privileges.

Remember: Escaping strings manually is fragile; parameterization does not replace authorization.

85. What is cross-site scripting?

Cross-site scripting occurs when attacker-controlled content executes in another user's browser in a trusted site's context. Use context-appropriate output encoding, safe rendering APIs, and vetted sanitization when permitting rich HTML.

Remember: Content Security Policy is a useful extra layer, not a substitute for correct rendering boundaries.

86. What is cross-site request forgery?

Cross-site request forgery causes a browser to send an unwanted authenticated request to a site, often using automatically attached cookies. Defenses include request tokens, suitable SameSite cookie settings, and origin checks where appropriate.

Remember: A request that changes state should not rely only on the fact that a session cookie is present.

87. What can and cannot be assumed about a JSON Web Token?

A signed JSON Web Token protects the integrity of its claims when validated correctly. Its payload is usually readable unless separately encrypted. Validate the expected issuer, audience, algorithm, expiry, and other application requirements.

Remember: Decoding is not validation; possession of a token can grant privileges until it expires or is revoked.

88. What is the principle of least privilege?

Give users, services, and processes only the access needed for their tasks, for only as long as needed. Apply it to database roles, cloud permissions, service identities, and operational tools.

Remember: Separate production and development privileges and review permissions as responsibilities change.

89. How should secrets be managed?

Keep credentials out of source code and logs. Use a secrets-management system or another controlled injection mechanism, limit access, rotate credentials, and audit usage. Treat accidentally committed secrets as compromised even if the commit is later removed.

Remember: Deleting a secret from the latest file does not remove it from history or existing copies.

90. How would you assess an unfamiliar application's security?

Identify valuable assets, entry points, trust boundaries, identities, and likely misuse cases. Review authentication, authorization, input handling, dependency exposure, secrets, and logging. Prioritize findings by credible impact and exploitability.

Remember: Threat modeling is a structured investigation, not a promise that every possible vulnerability has been found.

10 Testing, Git, and Engineering Practice

Questions 91-100

91. How do unit, integration, and end-to-end tests differ?

Unit tests isolate small behaviors. Integration tests verify boundaries between components. End-to-end tests exercise user-visible flows through a deployed or representative system. A good test mix balances speed, diagnostic value, realism, and maintenance.

Remember: The definitions depend on test boundaries; a test's filename does not determine its level.

92. What is regression testing?

Regression testing checks that previously working behavior still works after a change. Choose cases around the changed behavior, adjacent features, and critical invariants. Reproduce a fixed defect with a meaningful test when appropriate.

Remember: Passing regression tests only covers the behaviors and environments actually exercised.

93. Which edge cases should be tested in a coding interview?

Consider empty and single-element inputs, duplicates, extreme values, invalid inputs under the contract, and already sorted or reverse-sorted data. For graphs, check disconnected components, cycles, and self-loops when permitted.

Remember: Derive cases from the algorithm's assumptions instead of memorizing one universal checklist.

94. What makes a test reliable?

A reliable test has clear assertions, controlled dependencies, predictable data, and appropriate synchronization. Avoid depending on arbitrary sleep durations, execution order, or shared mutable state. Investigate flaky behavior rather than normalizing repeated reruns.

Remember: Prefer waiting for a meaningful state or event over guessing how many milliseconds to sleep.

95. How do Git merge and rebase differ?

A merge combines histories while retaining their structure, sometimes creating a merge commit. Rebase replays commits onto a new base, usually creating new commit identities. Coordinate before rewriting history others may be using.

Remember: Both approaches can require conflict resolution; a linear history does not mean conflicts never happened.

96. How do Git reset and revert differ?

Revert creates a new commit that undoes another commit's changes. Reset moves a reference and, depending on the mode, can update the index or working tree. Revert is often more suitable for undoing shared commits without rewriting history.

Remember: A hard reset can discard uncommitted tracked-file changes; inspect the intended target first.

97. What do continuous integration and continuous delivery mean?

Continuous integration frequently combines changes and runs automated validation. Continuous delivery keeps software in a releasable state with a controlled release decision. Continuous deployment automatically releases eligible changes to production.

Remember: A pipeline is only useful when its checks represent real product risks and failures are addressed.

98. How would you debug a production incident?

Establish impact and a timeline, inspect recent changes and relevant telemetry, and form testable hypotheses. Mitigate safely when needed, then investigate the cause and verify recovery. Record what happened and improve prevention or detection.

Remember: Correlation with a deployment is a clue, not proof of causation.

99. How do you optimize performance responsibly?

Define the workload and target, measure a baseline, profile the bottleneck, change one justified factor, and compare results. Examine latency distributions, throughput, memory, and correctness rather than optimizing one appealing microbenchmark.

Remember: An average can hide slow tail requests; a faster result that changes semantics is not a valid optimization.

100. How should a strong interview answer be structured?

Clarify the question, state assumptions, explain the core idea, and give a small example. Discuss complexity or tradeoffs, test edge cases, and acknowledge limits. For project questions, separate your contribution from team outcomes.

Remember: A concise, correct explanation with explicit assumptions is stronger than confidently listing unrelated terminology.

Further reading

Use the official documentation for your actual language version, database, and deployment. The Java tutorial listed here reflects JDK 8-era material. These original notes summarize concepts; the resources below support deeper study and implementation-specific details.

Python data structures

https://docs.python.org/3/tutorial/datastructures.html

Python data model

https://docs.python.org/3/reference/datamodel.html

Java interfaces and inheritance

https://docs.oracle.com/javase/tutorial/java/IandI/index.html

PostgreSQL transaction isolation

https://www.postgresql.org/docs/current/transaction-iso.html

HTTP semantics, RFC 9110

https://www.rfc-editor.org/rfc/rfc9110.html

SQL injection prevention, OWASP

https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

Git branching and rebasing

https://git-scm.com/book/en/v2/Git-Branching-Rebasing

Operating Systems: Three Easy Pieces

https://pages.cs.wisc.edu/~remzi/OSTEP/

MIT Introduction to Algorithms

https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/

Final revision routine

Explain ten questions per day. Revisit difficult answers the next day, then again after several days. Pair the algorithm topics with coding practice and the design topics with a small worked example.